

Mocking C#

SWEN1 | BIF3

Table of Contents

- Software Engineering – Mocking – C#



Software Engineering – Mocking – C#

Software Engineering – Mocking – C#

- Under test objects should be tested independently of other objects – „in isolation“
- When „real“ objects are very complex to instantiate and/or configure
- When only interfaces exist and implementations are not even coded

For this exercise we’ll use NSubstitute framework.

Software Engineering - Unit Testing - C#

Demo Project

- Create fighters with a lightsaber and a specific saber color
- Let the fighters fight with specific aims
- Combat and game mechanics to decide who won the fight



State vs. Interaction testing

- State testing evaluates properties on an object

```
Assert.That(fighter.Lightsaber.Color, Is.EqualTo(LightsaberColor.Blue));
```

- Interaction testing verifies the interactions between objects – the object's behavior

```
combat.GameMechanics(Aim.Attack, Aim.Rest);
mockedA.Received(0).ChangeVitality(0);
mockedB.Received(1).ChangeVitality(-2);
```

A friendly substitute for .NET mocking libraries



https://nsubstitute.github.io/

What is NSubstitute?

- NSubstitute is a .Net framework allowing the creation of test double objects (mock objects) in automated unit tests
- Clean & simple API – tests are very readable
- Enables mock creation, verification and stubbing
- For interaction testing

https://nsubstitute.github.io/help/getting-started/

Test Doubles

Test Double is a generic term for any case where you replace a production object for testing purposes

- Test Stub: hand coded object used for testing
- Mock Object
  - Expectations configured before calling the method under test,
    f.e. EasyMock
- Test Spy
  - Verification occurs after the method under test has been called, f.e. NSubstitute

Adding NSubstitute to the project

Install NuGet Package NSubstitute



NSubstitute by Anthony Egerton,David Tchepak,Alexandr Nikitin,Oleksandr Povar
NSubstitute is a friendly substitute for .NET mocking libraries. It has a simple, succinct syntax to help developers write clearer tests. NSubstitute is designed for Arrange-Act-Assert (AAA) testing and with Test Driven Developm



NSubstitute.Analyzers.CSharp by Tomasz Podolak, NSubstitute.Analyzers contributors
Provides diagnostic analyzers to warn about incorrect usage of NSubstitute in C#.



NUnit by Charlie Poole, Rob Prouse
NUnit is a unit-testing framework for all .NET languages with a strong TDD focus.



NUnit3TestAdapter by Charlie Poole, Terje Sandstrom
NUnit3 adapter for running tests in Visual Studio and DotNet. Works with NUnit 3.x, use the NUnit 2 adapter for 2.x tests.

First source sample

```
[Test]
public void TestMove_Attack2Attack()
// arrange
var mockedA = Substitute.For<IFighter>(); // mock
var mockedB = Substitute.For<IFighter>(); // mock
var combat = new Combat(mockedA, mockedB); // no mock

// act
combat.GameMechanics(Aim.Attack, Aim.Attack);
// the game-mechs for one round is applied

// assert
mockedA.Received(1).ChangeVitality(-1);
mockedB.Received(1).ChangeVitality(-1);
```

Working with mocks

- Once created, a mock will remember all interactions.
- Then you can selectively verify whatever interaction you are interested in.

- By default, for all methods that return value,
  mock returns null, an empty collection or appropriate primitive/primitive wrapper value

Sample with setup

```
public class CombatTest
{
  IFighter mockedA = Substitute.For<IFighter>(); // mock
  IFighter mockedB = Substitute.For<IFighter>(); // mock
  Combat? combat; // no mock

  [SetUp]
  public void SetUp()
  {
    // arrange
    combat = Substitute.For<Combat>(mockedA, mockedB);
  }

  [Test]
  public void TestMove_Attack2Rest()
  {
    // act
    combat?.GameMechanics(Aim.Attack, Aim.Rest);
    // assert
    mockedA.Received(0).ChangeVitality(0);
    mockedB.Received(1).ChangeVitality(-2);
  }
}
```

Verify behavior

- mock.Received(1).someMethod();
  → check if someMethod was called (once)
- mock.Received(0).someMethod();
  → ensure that someMethod was never called
- mock.Received(3).someMethod();
  → checks that someMethod was called 3 times.

Stubbing

- mock.someMethod().Returns(0);
  → when someMethod is called afterwards,
  → then the mock will return 0
- mock.someFunc(1,2).Return(3);
  → when someFunc will be called with params 1, 2
  → then the mock will return 3

Software Engineering – Mocking – C#

